

Definition 3 (Hidden-State Consistency). *An agent satisfies hidden-state consistency if, for some fixed secret $s \in D$, its replies at every round t equal $\mathcal{R}(s, H_{t-1}, x_t)$.*

Definition 4 (Public Underdetermination). *An agent satisfies public underdetermination if, for every round t strictly before the secret s is uniquely determined by the task rules, there exist $s' \neq s \in D$ that remain compatible with H_t . Equivalently, the public history H_t does not uniquely determine the secret s .*

Theorem 1 (Impossibility of Public Underdetermination and Hidden-State Consistency for POCAs). *No POCA can simultaneously guarantee both public underdetermination and hidden-state consistency in a PSIT.*

Lemma 1 (Non-implementability of PSIT under public-only chat). *Let $\mathcal{T}$ be any PSIT with secret $s \in D$ and deterministic reply rule $\mathcal{R}$. Then no POCA can satisfy both Public Underdetermination and Hidden-State Consistency simultaneously for all rounds whenever $|D| \geq 2$.*

Proof. Fix a POCA π and any user strategy that ensures at least two candidates remain compatible with the history before the final reveal. Let

$$C_{t-1} = \{w \in D : w \text{ is compatible with } H_{t-1}\}$$

be the candidate set at round $t - 1$. Suppose $|C_{t-1}| \geq 2$. Because C_{t-1} contains at least two distinct secrets, there exists a user input x_t and two candidates $s, s' \in C_{t-1}$ such that

$$\mathcal{R}(s, H_{t-1}, x_t) \neq \mathcal{R}(s', H_{t-1}, x_t).$$

By Hidden-State Consistency, if the true secret is s then $y_t = \mathcal{R}(s, H_{t-1}, x_t)$, while if the true secret is s' then $y_t = \mathcal{R}(s', H_{t-1}, x_t)$, with the two values differing by construction. However, since both s and s' are compatible with H_{t-1} , the POCA's reply distribution is the same in both worlds:

$$y_t \sim \pi(\cdot \mid H_{t-1}, x_t).$$

Thus the same output distribution must equal two different deterministic targets, which is impossible. Therefore, whenever Public Underdetermination holds ($|C_{t-1}| \geq 2$), Hidden-State Consistency must fail for some secret. Conversely, enforcing Hidden-State Consistency requires revealing information about s in the public history, violating Public Underdetermination.

Randomization does not help: $\pi(\cdot \mid H_{t-1}, x_t)$ is identical across indistinguishable secrets, but Hidden-State Consistency requires disjoint support across them. Hence no POCA can satisfy both properties simultaneously. $\square$

Implications for Existing Agents. The lemma directly applies to existing classes of LLM agents. Reasoning models operating under the standard chat interface can generate a secret within their internal reasoning trace, but this trace is discarded after each turn and therefore cannot serve as a persistent hidden state (OpenAI, 2025b;a; Hugging Face, 2025). RAG systems also fail, because their prompt augmentation draws from external, pre-existing sources (indexes, tools, databases) that are not privately or dynamically updated within the dialogue; as a result, the agent cannot generate and persist an in-dialogue secret. More generally, any framework that grounds its outputs only in public context or retrievable memory inherits the impossibility result, and therefore cannot satisfy both public underdetermination and hidden-state consistency in PSITs.

3 SELF-CONSISTENCY TESTING FOR PSITs

Having established formally that POCAs cannot both keep the secret unresolved in the public transcript and remain consistent with a fixed hidden state, we next turn to empirical evaluation. Our aim is to test whether, instead, memory-augmented agents can generate and maintain a hidden secret consistently while interacting in dialogue. To this end, we design a **self-consistency testing protocol** applied to two tasks, Hangman and the Diagnosis Simulator (described in the next paragraphs). The idea is to simulate controlled dialogues in which the agent needs to commit to a secret and respond according to task rules. To drive these interactions, we implement a deterministic, rule-based Player for each task that interacts with the agent according to a fixed policy, ensuring experimental control and reproducibility. The interaction begins with an opening turn where the Player defines the rules and invites the Agent to play, serving as the stimulus for the agent to internally generate and store the secret before questioning starts (e.g. "Let's play Hangman!"). At a chosen point (t_{fork}), we fork the conversation: in the first branch, the agent is asked to reveal its secret, and in the others it is asked yes/no questions about candidate secrets, all of which satisfy the constraints accumulated up to t_{fork} . This design enables us to assess whether the agent consistently affirms the revealed secret while simultaneously rejecting other equally plausible candidates.

Hangman. In this task, the agent acts as the host, privately selecting a secret word while the user guesses letters (see Appendix Figure 1 a). The Player issues guesses using a seeded policy that balances frequency-weighted sampling (prioritizing high-probability English letters like ‘e’, ‘t’, ‘a’) with exploration. After each guess, the agent must update the public state, displaying the current word pattern (e.g., “_ a _ e _”) and the list of guessed letters. At the fork point t_{fork} , we extract these hard constraints (word length, revealed letter positions, and absent letters) and filter the Wordfreq database (Speer, 2022) to generate a set of candidate words that are syntactically indistinguishable from the secret given the public history; if the database yields insufficient candidates, we supplement the set with LLM-generated proposals.

Diagnosis Simulator. This task evaluates consistency in the medical domain (see Figure 1 b), and it has been designed to emulate a medical student practicing differential diagnosis with an LLM. In the interaction the agent acts as the patient holding a hidden ground-truth condition, while the Player questions them to narrow down possibilities. Utilizing the DDXPlus dataset (Fansi Tchango et al., 2022), the Player issues yes/no questions about specific symptoms (evidences) to prune the hypothesis space. At t_{fork} , we generate the candidate set by filtering the dataset for all conditions that remain logically consistent with the accumulated positive and negative evidence (e.g., Fever=Yes, Cough=No). As before, this set is deduplicated and, if the database yields insufficient candidates, we supplement the set with LLM-generated proposals.

Evaluation. To assess the agent’s performance, we spawn one dialogue branch for each candidate secret and ask the agent to confirm or deny it (e.g., ‘Is the secret word “word”?’); we explicitly instruct the agent to output only ‘yes’ or ‘no’ to facilitate deterministic parsing. We first identify cases of **Leakage**, where the secret was explicitly revealed in the public transcript prior to the reveal turn (detected via case-insensitive matching). These are strictly categorized as failures of public underdetermination. For the remaining non-leaking runs, we measure **Self-Consistency**, defined as the rate at which the agent consistently affirms its revealed secret across branches without affirming any other candidate. We categorize the remaining failures into three behaviors: **Over-Confirmation**, where the agent correctly affirms the revealed secret but also falsely affirms at least one alternative candidate; **State Substitution**, where the agent denies the revealed secret but affirms at least a different candidate; and **All Denial**, where the agent rejects every proposed candidate.

The full prompt specifications for both tasks are provided in Appendix G.

3.1 COMPARATIVE BASELINES.

We compare our proposed methods against several baselines. We first consider Vanilla Stateless LLMs (standard POCAs), which generate responses solely from the public dialogue history. When using reasoning models (Section 4), we explicitly discard their internal reasoning traces to enforce the POCA setting. To establish an upper bound, we also evaluate Private Chain-of-Thought (CoT) agents that retain all hidden reasoning across turns. This baseline shows that persistent hidden traces are sufficient in principle to solve PSITs, but it is not a practical deployment solution: reasoning traces are highly verbose, causing the token budget to grow rapidly, precisely why standard chat interfaces discard them. Our aim is therefore to match this hidden-state benefit with a bounded and controllable memory representation. A cost analysis is provided in Section 5.

Agentic Memory Baselines. We compare against state-of-the-art memory-augmented agents that rely on external persistent textual memory to support generation. Although these systems are designed for long-term retrieval and conversational coherence rather than private state maintenance, they provide the closest existing baselines for evaluating whether current memory designs can implicitly support hidden state. In all cases, we use the baselines’ official libraries and native memory update and retrieval APIs, adapting only their interface to the shared dialogue setting.

A-mem organizes interactions into atomic, linked notes inspired by the Zettelkasten method, with LLM-driven semantic linking and memory updates (Kadavy, 2021; Xu et al., 2025).

Mem0 maintains long-term consistency via LLM-based extraction and dynamic ADD/UPDATE/DELETE operations over stored facts (Chhikara et al., 2025).

LightMem follows a hierarchical memory architecture with sensory, short-term, and long-term components to balance efficiency and retrieval quality (Atkinson & Shiffrin, 1968; Fang et al., 2025).

MemoryOS employs an operating-system-inspired hierarchical memory architecture with short-term, mid-term, and long-term storage tiers using a segmented page organization strategy with heat-based importance scoring (Kang et al., 2025).

3.2 PROPOSED PRIVATE WORKING MEMORY AGENTS.

Recent discussions have emphasized the importance of distinguishing between *workflows* and *agents* as architectural paradigms for LLM-based systems (see Appendix Figure 5). Both can be represented as directed acyclic graphs, but they differ in how control flows through the graph. In a workflow, the edges are fixed and predetermined: the LLM is invoked at specific nodes, but the overall orchestration is governed programmatically, making the system more predictable and controllable (Anthropic, 2024; Yu et al., 2025; Niu et al., 2025). In contrast, an agent introduces conditional branching, where the choice of which edge to follow is delegated to the LLM’s decision-making. This conditionality provides more flexibility, but at the cost of higher variance, and new failure modes. Thus, workflows are well-suited to structured, repeatable processes, while agents are preferable when open-ended reasoning and adaptation are required.

To enable private state in both paradigms, we introduce a textual working memory. Unlike vector-based retrieval, this memory consists of a text block that is injected into the system prompt (wrapped in `<private_state>` tags) at every turn. This state is persistent for the agent but invisible to the user. We evaluate two implementations of this mechanism: **Autonomous Memory Agent**, in which the LLM is provided with memory-management tools and autonomously decides whether and when to update its private state; **Memory Workflows**, in which the interaction is governed by a deterministic graph where the system forces a two-step execution per turn: the generation of the public response and a memory update step.

Memory Representation. We adopt a cognitively inspired design for private working memory, motivated by classic models of working memory and recent language-agent architectures. Specifically, the memory is structured into three functional components, goals/plans, salient facts, and active inferences, to support concise storage and effective decision-making across interaction steps. The agent is explicitly instructed on the role of each component and maintains them throughout the dialogue. Details on the design rationale and prompt specifications are provided in Appendix B and G.

Memory Update Strategies. For both proposed architectures (Autonomous and Workflow), we evaluate three private-state maintenance mechanisms to study their impact on agent performance; full tool definitions are provided in Appendix H:

Overwrite. The simplest strategy, where the agent generates the complete updated memory state at each step. The `overwrite_memory` tool takes a single argument, the new memory string, and fully replaces the existing content, without parsing or structural constraints.

Append/Delete. A section-aware strategy using two complementary operations. The working memory is parsed into numbered sections (e.g., “## 1. Goals and Plans”), and the agent operates on specific sections by name. The `append_in_memory` tool adds new lines to the end of a designated section, while `delete_from_memory` removes entries using robust fuzzy matching. For longer targets (≥ 8 characters), substring matching is permitted; shorter targets require exact canonical equality to prevent accidental broad deletions.

Patch/Replace-in. A fine-grained strategy inspired by industrial code editing tools such as the open-source VS Code Copilot Chat framework (GitHub, 2025). Two tools are provided: `patch_memory` accepts a diff-like format with section anchors (`@@ section: <Title>`), deletion lines prefixed with “-”, and insertion lines prefixed with “+”, with optional context lines for disambiguation. The `replace_in_memory` tool performs surgical find-and-replace operations, specifying an exact `old_string` to locate and a `new_string` to substitute. Both tools support optional pre- and post-context anchors to resolve ambiguity when the target text appears multiple times and safety parameters (`expected_hunks`, `expected_replacements`) that cause the operation to fail if the actual match count differs from intent. Both tools are idempotent: re-applying an already-applied edit produces no change.

This progression from full replacement, to selective editing, to fine-grained patching, represents increasingly flexible ways of managing memory, allowing us to examine how the complexity of updates affects whether agents can keep their secrets, stay consistent, and play the tasks correctly.

4 EXPERIMENTAL SETUP

We evaluate our approach across two state-of-the-art open-weights model families: **GPT-OSS** (20B and 120B) (Agarwal et al., 2025) and **Qwen3** (32B and 235B) (Yang et al., 2025). We selected these models because they are reasoning models, required for the Private CoT baseline, and possess tool-use capabilities, necessary for memory-augmented baselines and our architectures.